Chronic Disease Risk Prediction — Architecture & Code Walkthrough

1. System Overview

What this feature does, in plain language

When a clinician opens a patient's consultation page, the SES system automatically calculates how likely that patient is to develop four chronic diseases: **Hypertension**, **Cardiovascular Disease**, **Diabetes**, and **Chronic Kidney Disease (CKD)**. It does this by pulling together everything the EHR knows about the patient — their blood pressure history, lab results, medications, and how often they attend clinic — running that data through four trained machine learning models, and displaying a colour-coded risk score alongside an explanation of which factors are driving that risk most strongly.

The clinician sees a panel labelled "Chronic Disease Risk" showing something like:

```
● Hypertension    72%   [ High   ]  ▸
● Cardiovascular  41%   [ Moderate]  ▸
● Diabetes        14%   [ Low    ]  ▸
● CKD             58%   [ High   ]  ▸
```

Clicking any row expands it to show the top three contributing factors — for example "Blood pressure of 148 mmHg is the primary driver of risk" — so the clinician can understand *why* the model produced that number, not just what the number is.

The problem this solves

Chronic diseases develop over years. A single blood pressure reading does not tell you as much as knowing the patient's average over six months and whether it is trending upward. A clinician examining a patient for a headache may not have time to manually pull together years of vitals, labs, and visit history to assess metabolic risk. This feature does that aggregation automatically and surfaces the risk score passively — it appears without any extra action required.

The three stages data passes through

```
Stage 1: PATIENT DATA ASSEMBLY (Node.js / Express)
  - Reads from the PostgreSQL database
  - Pulls vitals, labs, medications, encounters
  - Computes derived values (BMI, averages, trends)
  - Assembles one 54-field object called the "feature vector"

Stage 2: RISK PREDICTION (Python / FastAPI)
  - Receives the feature vector over HTTP
  - Passes it through four XGBoost models
  - Computes SHAP values to explain each score
```

```
    - Writes a plain-language clinical summary
    - Returns JSON with four predictions

  Stage 3: DISPLAY (React / TypeScript)
    - Calls Stage 1 via an HTTP GET request
    - Receives the Stage 2 result forwarded through Stage 1
    - Renders colour-coded badges, percentages, and expandable factor cards
```

Folder / file map — where each piece physically lives in the repo

```
SES/
├── client/                          ← React frontend (TypeScript)
│   └── src/
│       └── components/
│           └── consultation/
│               └── CdsPanel/
│                   └── CdsPanel.tsx  ← Renders the risk section in the UI
│
├── server/                          ← Express backend (Node.js / TypeScript)
│   └── src/
│       ├── routes/
│       │   └── index.ts             ← Declares GET /patients/:id/risk-predict
│       └── controllers/
│           └── risk.controller.ts   ← Queries DB, assembles feature vector,
│                                          forwards to FastAPI
│
└── cds-service/                     ← FastAPI backend (Python)
    ├── main.py                      ← App entry point; mounts risk_router
    ├── risk_router.py               ← POST /cds/risk-predict endpoint
    ├── risk_model.py                ← Model loading + predict() function
    └── models/
        ├── feature_cols.json        ← Ordered list of 54 feature names
        ├── model_hypertension.json  ← XGBoost model file
        ├── model_cvd.json
        ├── model_diabetes.json
        └── model_ckd.json
```

Why is this split across two separate backend services?

A natural question: why does the Node.js server exist at all for this feature? Why not have the React app call the Python service directly, or have Python query the database itself?

The Node server handles the database because that is its job throughout the whole application. Every other feature — appointments, prescriptions, lab orders, billing — already goes through the Express server and its Prisma ORM layer. The database schema, authentication middleware, and business logic all live there. It would be poor design to bypass this and let the Python service make its own PostgreSQL connections; it would duplicate the authentication layer, duplicate the schema knowledge, and create two independent connections to the same database.

The Python service handles the models because JavaScript cannot. XGBoost is a Python-native library. The models were trained in Python using `xgboost`, `scikit-learn`, and `shap`. Running them in Node would require either a subprocess call (fragile), a WASM port (not production-ready), or rewriting the training pipeline in JavaScript (impractical). Python is simply the right tool for this specific task.

So the split is: **Node owns the data**; **Python owns the models**. They communicate over a single HTTP call between the two services.

---

2. Request Walkthrough — Step by Step

This section traces a single request from the moment a clinician opens a consultation page to the moment the risk scores appear on screen, showing the actual code at each step and explaining exactly what it does.

Step 1 — React triggers the request

File: `client/src/components/consultation/CdsPanel/CdsPanel.tsx`

Why this file? `CdsPanel` is the right panel in the consultation layout. It already owns the guideline-based CDS panel. Adding risk prediction here follows the same pattern: a collapsible section that loads automatically when the panel mounts, displays a result, and refreshes when the clinician clicks the refresh button. Placing it in this component avoids creating a separate component for something that is visually part of the same panel.

The relevant code starts with the state declarations:

```
// Declared inside the CdsPanel component function
const [riskCollapsed, setRiskCollapsed]           = useState(false);
const [riskData, setRiskData]                     = useState<RiskPredictions |
null>(null);
const [riskLoading, setRiskLoading]               = useState(false);
const [riskError, setRiskError]                   = useState<string | null>(null);
const [expandedConditions, setExpandedConditions] = useState<Set<string>>(new
Set());
```

What this does: These are React state variables. `useState` is React's way of storing data that can change over time and cause the screen to re-render when it does. `riskData` will hold the JSON that comes back from the server — it starts as `null` meaning "not yet loaded". `riskLoading` is a flag that is `true` while the request is in flight, used to show the pulsing dot on the header. `expandedConditions` is a `Set` (a collection of unique strings) tracking which conditions the user has expanded to see the SHAP factors.

The function that actually makes the network call:

```
const fetchRisk = useCallback(async () => {
  if (!isMounted.current) return;       // Don't update state after component
unmounts
  setRiskLoading(true);
  setRiskError(null);
  try {
```

```
      const { data } = await api.get(`/patients/${patientId}/risk-predict`);
      if (isMounted.current) setRiskData(data.predictions as RiskPredictions);
    } catch (err: any) {
      if (isMounted.current)
        setRiskError(err?.response?.data?.error ?? 'Risk prediction service
unavailable');
    } finally {
      if (isMounted.current) setRiskLoading(false);
    }
  }, [patientId]);
```

What this does, line by line:

- `useCallback` — This wraps the function so React does not re-create it on every render, only when `patientId` changes. This matters because the function is used as a dependency in another hook below.
- `if (!isMounted.current) return` — If the user navigates away before the request finishes, this prevents calling `setRiskData` on an unmounted component (which would cause a React error).
- `api.get(...)` — `api` is an Axios instance configured with `baseURL: '/api'`. So `api.get('/patients/${patientId}/risk-predict')` sends a GET request to `/api/patients/6c701205.../risk-predict`. Vite's development proxy intercepts requests starting with `/api` and forwards them to `http://localhost:5000`, so the Express server receives this.
- `data.predictions` — The full JSON response has shape `{ patientId, status, predictions, features }`. The component only needs `predictions`, so it pulls just that field.
- The `catch` block — If anything goes wrong (network error, server error, FastAPI down), it stores the error message in `riskError`. The `??` operator means: use `err.response.data.error` if it exists, otherwise use the fallback string.

The hook that triggers the fetch automatically:

```
useEffect(() => { fetchRisk(); }, [fetchRisk]);
```

What this does: `useEffect` runs side effects — things that happen *because of* a render rather than *during* a render, like network requests. The `[fetchRisk]` in brackets is the dependency array. This effect runs whenever `fetchRisk` changes. `fetchRisk` is recreated by `useCallback` only when `patientId` changes. So in practice: the risk data is fetched once when the component first mounts (on page load), and again whenever the patient changes (navigating to a different patient's consultation page).

The refresh button wires both pipelines together:

```
function handleRefresh() {
  trigger(true);   // Re-run the guideline CDS AI (existing feature)
  fetchRisk();     // Re-run the ML risk prediction (new feature)
}
```

What this does: When the clinician clicks the refresh button, both the guideline-based CDS and the ML risk scores are re-fetched. This is intentional — the clinician may have just recorded new vitals or added a lab

result, and both systems should reflect the updated data.

The rendering code for each condition row:

```
{CONDITION_ORDER.map(key => {
  const cond = riskData[key];              // e.g. riskData["hypertension"]
  const rs   = RISK_STYLE[cond.risk_level]; // look up colours for "high", "low",
etc.
  const pct  = Math.round(cond.probability * 100); // 0.72 → 72

  return (
    <button onClick={() => toggleCondition(key)} ...>
      <span style={{ background: rs.dot }} />  {/* coloured dot */}
      <span>{CONDITION_LABELS[key]}</span>       {/* "Hypertension" */}
      <span>{pct}%</span>                        {/* "72%" */}
      <span style={{ background: rs.bg, color: rs.text }}>{rs.label}</span>
    </button>
  );
})}
```

What this does: For each of the four conditions (in the order defined by `CONDITION_ORDER: ['hypertension', 'cardiovascular', 'diabetes', 'ckd']`), it looks up the model's prediction, converts the raw decimal probability to a percentage, and renders a row with a coloured indicator. `RISK_STYLE` is a lookup table mapping the string risk level to CSS colour values:

```
const RISK_STYLE = {
  'low':       { bg: 'var(--badge-green-bg)', text: 'var(--badge-green-text)',
dot: '#22c55e', label: 'Low'       },
  'moderate':  { bg: 'var(--badge-amber-bg)', text: 'var(--badge-amber-text)',
dot: '#eab308', label: 'Moderate'  },
  'high':      { bg: '#fff7ed',               text: '#c2410c',
dot: '#f97316', label: 'High'      },
  'very high': { bg: 'var(--badge-red-bg)',   text: 'var(--badge-red-text)',
dot: '#ef4444', label: 'Very High' },
};
```

The `var(--badge-green-bg)` notation uses CSS custom properties (variables) defined in `index.css` for the application's theme, so colours adapt correctly in both light and dark mode without the component needing to know which theme is active.

Step 2 — The request arrives at Express

File: `server/src/routes/index.ts` (line 126)

```
router.get('/patients/:patientId/risk-predict', authenticate, getRiskPrediction);
```

What this does: This single line registers the HTTP route. When Express receives `GET /patients/6c701205.../risk-predict`, it first calls `authenticate` (which checks the JWT token in the Authorization header), and if authentication passes, it calls `getRiskPrediction` (defined in `risk.controller.ts`).

Why it's in this file: `index.ts` is the central route registry — every API endpoint in the entire Express server is declared here. This is a deliberate architectural choice: any developer can open one file to see all routes, their authentication status, and which controller handles them. Adding a new endpoint requires a single line here and a new function in a controller file.

What `authenticate` does: It reads `req.headers.authorization`, extracts the Bearer token, and verifies it against `JWT_SECRET`. If the token is missing, expired, or tampered with, it returns `401 Unauthorized` immediately and `getRiskPrediction` is never called. The clinician must be logged in.

Step 3 — The controller assembles the feature vector

File: `server/src/controllers/risk.controller.ts`

Why is this a "controller" and what does that mean? In the Express codebase, a "controller" is a file that contains the actual logic for handling a specific set of HTTP endpoints. The `routes/index.ts` file only *declares* what URL maps to what function; the controller file *implements* what that function does. This separation keeps routing simple (routes is just a list) and keeps logic organised (each controller owns a domain — `auth.controller.ts` handles logins, `pharmacy.controller.ts` handles medications, `risk.controller.ts` handles risk prediction). If you want to understand what happens when a risk endpoint is called, you open the controller — not the routes file.

The controller's single exported function:

```
export async function getRiskPrediction(req: Request, res: Response):
Promise<void> {
  const patientId = String(req.params['patientId']);
  console.log(`[risk] GET /patients/${patientId}/risk-predict`);
```

`req` and `res` are Express conventions. `req` (request) contains everything about the incoming HTTP call — URL parameters, headers, body, authentication info. `res` (response) is the object you call methods on to send data back to the caller. `async` means this function contains `await` calls that pause execution while waiting for database queries or network calls. `String(req.params['patientId'])` converts the URL segment to a guaranteed string — in Express v5, URL parameters can theoretically be `string | string[]`, and this conversion makes the intent explicit.

Database query 1 — Patient demographics and medications:

```
const patient = await prisma.patient.findUnique({
  where: { id: patientId },
  select: {
    dateOfBirth: true,
    gender:      true,
```

```
    hivStatus:   true,
    medications: { select: { name: true } },
  },
});
```

What this does: Prisma is the database client — it translates this JavaScript object into a SQL `SELECT` statement and returns typed results. `findUnique` means "find exactly one record where the `id` column matches". The `select` block is explicit about which columns to return; rather than fetching the entire patient row, it only asks for the four fields needed. The `medications` entry is a *relation select*: Prisma follows the foreign key relationship to the `Medication` table and returns every medication row linked to this patient, but only the `name` column of each.

```
if (!patient) {
  res.status(404).json({ error: 'Patient not found' });
  return;
}
```

If the patient ID does not exist in the database, the function immediately returns a 404 error and stops. No further processing happens.

Encoding demographics:

```
const age = ageYears(patient.dateOfBirth);

const gender: number | null =
  patient.gender === 'MALE' ? 1 : patient.gender === 'FEMALE' ? 0 : null;

const hiv_status: number | null =
  patient.hivStatus === 'POSITIVE' ? 1 :
  patient.hivStatus === 'NEGATIVE' ? 0 : null;
```

What this does: The database stores `gender` as the string `'MALE'` or `'FEMALE'` (a PostgreSQL enum called `Gender`). The XGBoost models were trained with gender as a *number* — 1 for male, 0 for female. The controller converts here. `null` represents genuinely unknown or other values, and XGBoost handles `null` gracefully as missing data (discussed more in section 3).

The same pattern applies to HIV status: the database stores `'POSITIVE'`, `'NEGATIVE'`, or `'UNKNOWN'`; the model expects `1`, `0`, or missing.

```
const medNames = ((patient as any).medications as Array<{ name: string }> ?? [])
  .map((m) => m.name.toLowerCase());

const on_art: number | null =
  hiv_status === 1 && medNames.some((n) => ART_KEYWORDS.some((kw) =>
```

```
  n.includes(kw)))
      ? 1 : null;
```

What this does: `ART_KEYWORDS` is a list of antiretroviral drug names (`'efavirenz'`, `'tenofovir'`, `'dolutegravir'`, etc.). `medNames.some((n) => ART_KEYWORDS.some((kw) => n.includes(kw)))` asks: does any of this patient's medication names contain any of the ART keyword strings? The entire check is only meaningful if the patient is HIV positive (`hiv_status === 1`). For all other cases, `on_art` is `null` — we don't know. The `(patient as any).medications ?? []` is a defensive guard: if the Prisma client is outdated (before `medications` was added as a relation), it prevents a crash by falling back to an empty array.

Database query 2 — Vital signs time series:

```
const vitals = await prisma.vital.findMany({
  where: { patientId },
  orderBy: { recordedAt: 'asc' },
});
```

What this does: Fetches every vital signs record for this patient, ordered oldest to newest. The ordering matters: the time-series features below depend on the sequence of readings.

Computing blood pressure features:

```
const systolics  = vitals.map((v) => v.bloodPressureSystolic)
                         .filter((v): v is number => v != null);
const sys6m      = vitals
  .filter((v) => v.recordedAt >= ago6m && v.bloodPressureSystolic != null)
  .map((v) => v.bloodPressureSystolic as number);
const sys12m     = vitals
  .filter((v) => v.recordedAt >= ago12m && v.bloodPressureSystolic != null)
  .map((v) => v.bloodPressureSystolic as number);
const sysLast3   = systolics.slice(-3);

const bp_reading1    = systolics.at(-1) ?? null;           // most recent
const bp_reading2    = vitals.at(-1)?.bloodPressureDiastolic ?? null;
const bp_visit_avg   = avg(systolics);                     // all-time average
const bp_3visit_avg  = avg(sysLast3);                      // last 3 readings
const bp_6month_avg  = avg(sys6m);                         // 6-month average
const bp_12month_max = sys12m.length ? Math.max(...sys12m) : null;
const bp_trend       = linSlope(systolics);
const bp_variability = stddev(systolics);
const elevated_visit_count = systolics.filter((s) => s >= 140).length || null;
```

What this does, feature by feature:

- `systolics` — extracts just the systolic (top number) readings and removes any `null` entries (vitals where BP was not recorded that day).
- `bp_reading1` — the single most recent systolic reading. `.at(-1)` is the last-element accessor.

- `bp_reading2` — the diastolic (bottom number) from the most recent vital record. Note: this is from the most recent record overall, not necessarily the most recent systolic, because `vitals.at(-1)` refers to the last row in the time-ordered array.
- `bp_visit_avg` — average of all systolic readings ever recorded. The `avg()` helper function sums the array and divides by its length.
- `bp_3visit_avg` — average of just the three most recent systolic readings. `slice(-3)` returns the last three elements of the array.
- `bp_6month_avg` — average of systolics recorded in the last 6 months. `ago6m` is computed as `new Date(now.getTime() - 180 * 86_400_000)` — current time minus 180 days in milliseconds.
- `bp_12month_max` — the highest single systolic reading in the last 12 months. `Math.max(...sys12m)` spreads the array as arguments to `Math.max`.
- `bp_trend` — the slope of a straight line fitted to the systolic readings over time. The `linSlope()` function implements linear regression: if the line slopes upward (positive number), BP is trending up; if negative, it is improving. The maths: each reading is assigned an x-coordinate (0, 1, 2, ...) representing its position in time. The slope formula is `Σ((x - x̄)(y - ȳ)) / Σ((x - x̄)²)`.
- `bp_variability` — the standard deviation of all systolic readings. High variability (readings that bounce between 90 and 160) is independently associated with cardiovascular risk.
- `elevated_visit_count` — how many individual readings were ≥140 mmHg (the clinical threshold for hypertension). `|| null` converts a count of zero to `null` because zero is not meaningfully different from "no data" for the model.

BMI calculation:

```
let patientBmi: number | null = null;
for (let i = vitals.length - 1; i >= 0; i--) {
  const b = bmi(vitals[i].weight, vitals[i].height);
  if (b !== null) { patientBmi = b; break; }
}
```

What this does: Walks backward through the vitals list (most recent first) to find the first record that has both weight and height. The `bmi()` helper:

```
function bmi(weight?: number | null, height?: number | null): number | null {
  if (!weight || !height || height <= 0) return null;
  return parseFloat((weight / (height / 100) ** 2).toFixed(1));
}
```

BMI is weight in kilograms divided by height in metres squared. Since heights in the database are stored in centimetres, `height / 100` converts to metres. The formula `(height / 100) ** 2` squares the result. `toFixed(1)` rounds to one decimal place, and `parseFloat` converts the string back to a number.

The eGFR (kidney function) calculation:

```
function ckdEpiEgfr(creatinine_umol: number, age: number, gender: number): number
{
```

```
  const scr = creatinine_umol / 88.4;              // µmol/L → mg/dL
  const kappa = gender === 1 ? 0.9 : 0.7;
  const alpha = gender === 1 ? -0.302 : -0.241;
  const sexFactor = gender === 1 ? 1.0 : 1.012;
  const ratio = scr / kappa;
  const egfr =
    142 *
    Math.pow(Math.min(ratio, 1), alpha) *
    Math.pow(Math.max(ratio, 1), -1.2) *
    Math.pow(0.9938, age) *
    sexFactor;
  return parseFloat(egfr.toFixed(1));
}
```

What this does: eGFR (estimated Glomerular Filtration Rate) measures how well the kidneys are filtering waste from the blood. A value above 90 is normal; below 60 indicates kidney impairment.

This implements the **CKD-EPI 2021 race-free formula** — the current international clinical standard. The formula uses creatinine level (a waste product the kidneys filter), age, and sex:

- `creatinine_umol / 88.4` converts from the EHR's units (micromoles per litre, µmol/L) to the formula's expected units (milligrams per decilitre, mg/dL).
- `kappa` and `alpha` are sex-specific constants from the formula specification. Males have kappa=0.9, females kappa=0.7.
- `Math.min(ratio, 1)` raised to `alpha` handles the case where creatinine is below the kappa threshold. `Math.max(ratio, 1)` raised to `-1.2` handles the above-threshold case. Together, these two terms create a piecewise calculation that is "kinked" at the kappa point.
- `Math.pow(0.9938, age)` — older patients naturally have lower eGFR; this term decreases eGFR by ~0.62% per year.
- `sexFactor` — 1.012 for females (the 2021 formula dropped the larger factor that previously existed for race, replacing it with this slight sex adjustment).

The investigation results and lab component ID mapping:

```
const COMPONENT_MAP: Record<number, ComponentMapping> = {
  53: { feature: 'fasting_glucose_mmol' },
  54: { feature: 'rbs_mmol' },
  62: { feature: 'hba1c_pct' },
  31: { feature: 'creatinine_umol', convert: (v) => v * 1000 },
  32: { feature: 'egfr' },
  2:  { feature: 'haemoglobin_g_dl' },
  // ... (16 entries total)
};
```

What this does: Investigation results in the EHR are stored as JSON blobs containing component IDs — integer codes from the test catalogue. For example, a full blood count order might produce results like:

```
[
  { "componentId": 2,  "value": "16",   "units": "g/dL" },
  { "componentId": 11, "value": "7.2",  "units": "x10⁹/L" }
]
```

`COMPONENT_MAP` is a lookup table that translates catalogue IDs (which are arbitrary numbers chosen by the investigation catalogue designer) to the named features the model expects. ID `2` is haemoglobin; ID `11` is white blood cells.

The optional `convert` function handles unit mismatches. Creatinine (ID 31) is stored in the EHR in mmol/L, but the XGBoost model was trained on data measured in µmol/L. The conversion is `v * 1000` (multiply by 1000). Similarly, uric acid (ID 60) is stored in mg/dL but the model expects µmol/L, so `v * 59.48` applies the molecular weight conversion.

The query to fetch investigation results:

```
const orders = await prisma.investigationOrder.findMany({
  where: { patientId },
  include: {
    items: {
      include: {
        results: { orderBy: { resultedAt: 'asc' } },
      },
    },
  },
});
```

What this does: Investigation orders have a three-level hierarchy:

- `InvestigationOrder` — a request placed by a clinician (e.g., "do a full blood count")
- `InvestigationOrderItem` — one test within that request (e.g., "haemoglobin")
- `InvestigationResult` — the actual reported value, timestamped

`include` tells Prisma to follow these relationships and return them nested. The result is an array of orders, each containing an array of items, each containing an array of results.

Extracting the most recent value per component:

```
const latestByComponentId = new Map<number, Reading>();

for (const order of orders) {
  for (const item of (order as any).items ?? []) {
    for (const result of item.results) {
      let components: Array<{ componentId: number | string; value: string }>;
      try {
        components = JSON.parse(result.resultsJson);
      } catch {
```

```
        continue;
      }
      for (const comp of components) {
        const cid = Number(comp.componentId);
        if (!COMPONENT_MAP[cid]) continue;          // ignore unknown components
        const raw = parseFloat(String(comp.value));
        if (!isFinite(raw)) continue;               // ignore text values like
"Positive"
        const existing = latestByComponentId.get(cid);
        if (!existing || result.resultedAt > existing.ts) {
          latestByComponentId.set(cid, { raw, ts: result.resultedAt });
        }
      }
    }
  }
}
```

What this does, line by line:

- `new Map<number, Reading>()` — a dictionary keyed by component ID, storing the most recent result for each. We use a Map rather than a plain object because the keys are numbers and Map is more efficient and semantically clearer for this use.
- The three nested `for` loops walk the entire investigation history: order → item → result.
- `JSON.parse(result.resultsJson)` — each result row stores its components as a JSON string in the database. We parse it back to an array.
- `if (!COMPONENT_MAP[cid]) continue` — skip components we have no model feature for. There are hundreds of possible test components in the catalogue; most are irrelevant to these four chronic disease models.
- `parseFloat(String(comp.value))` — values are stored as strings in the JSON (`"16"`, `"7.2"`). `parseFloat` converts them to numbers. A non-numeric string like `"Positive"` returns `NaN`.
- `if (!isFinite(raw)) continue` — `NaN` is not finite, so this line silently skips text-based results (qualitative tests like "Positive/Negative").
- `if (!existing || result.resultedAt > existing.ts)` — only keep the *most recent* value. If a patient has had haemoglobin tested three times, we use the latest result.

How missing data is handled:

At the end, a final features object is assembled:

```
const features: Record<string, number | null> = {
  age,
  gender,
  bmi:          patientBmi,
  waist_cm:     null,              // not captured in this EHR
  smoking_status: null,            // not captured
  // ...
  creatinine_umol,                 // null if no lab result on file
  egfr,                            // null if no creatinine
  haemoglobin_g_dl: labFeatures['haemoglobin_g_dl'] ?? null,
```

```
  // ...
};
```

Every field in the object is always present — but its value is either a JavaScript `number` or `null`. There are never strings, never `undefined`, never omitted keys. When serialised to JSON for the HTTP call to FastAPI, `null` becomes JSON `null`, which Python and XGBoost treat as "missing" (NaN internally). This is intentional: XGBoost trees can naturally handle missing values by learning which branch to take when a feature is absent.

Step 4 — The HTTP handoff from Node to FastAPI

```
const CDS_BASE = process.env['CDS_URL'] ?? 'http://localhost:8000';

console.log(`[risk] DB stage complete for ${patientId}, forwarding to FastAPI`);
console.log('[risk] features:', JSON.stringify(features, null, 2));

let cdsResponse: Response | globalThis.Response;
try {
  cdsResponse = await fetch(`${CDS_BASE}/cds/risk-predict`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(features),
  });
} catch (fetchErr: any) {
  res.status(502).json({ error: `Risk prediction service unreachable:
${fetchErr.message}` });
  return;
}
```

What this does: `fetch` is the standard HTTP client built into Node.js 18+ (no library import needed). It sends a `POST` request to FastAPI on port 8000, with the complete 54-field feature vector as the JSON body. `JSON.stringify` converts the JavaScript object to a string for transmission over HTTP. `await` pauses execution until FastAPI responds.

Why is this a network call rather than a function call? Node.js is JavaScript; the XGBoost model code is Python. They cannot directly call each other's functions. The two processes are completely separate operating system processes running concurrently. HTTP over localhost is the standard way to communicate between two processes on the same machine. `CDS_URL` can be overridden via an environment variable, which is how this would work in a cloud deployment where the services run on different machines or containers.

The response handling:

```
if (!cdsResponse.ok) {
  const body = await cdsResponse.text();
  res.status(502).json({
    error: `Risk prediction service error (${cdsResponse.status}): ${body}`
  });
  return;
```

```
  }

  const prediction = await (cdsResponse as globalThis.Response).json() as {
    status: string;
    predictions: Record<string, unknown>;
  };

  res.json({
    patientId,
    status:      prediction.status,
    predictions: prediction.predictions,
    features,    // returned for transparency — frontend can display which data was
  used
  });
```

What this does: `cdsResponse.ok` is true for HTTP 200-299 responses. If FastAPI returned an error (e.g., model loading failed), `cdsResponse.ok` is false and we send a 502 "Bad Gateway" error to the React client — informing the frontend that the upstream Python service had a problem, distinct from the Node server itself having a problem.

If successful, the Node server passes `prediction.predictions` straight through to the React client, along with the `features` object (the assembled data) for debugging and transparency purposes.

Step 5 — FastAPI receives and routes the request

File: `cds-service/main.py` (lines 362–366)

```
from risk_router import router as risk_router
app.include_router(risk_router)
```

What this does: FastAPI uses "routers" to organise endpoints across multiple files. `app.include_router(risk_router)` registers all routes defined in `risk_router.py` into the main application. This happens at startup, before any request is ever received.

Why is this in `main.py` rather than directly in `risk_router.py`? `main.py` is the entry point — the file that Uvicorn (the ASGI server running FastAPI) starts. It creates the `app` object. All routers must be registered on that single `app`. If risk prediction logic lived directly in `main.py`, the file would grow to thousands of lines as more features are added. Instead, each feature area (document processing, risk prediction, the main CDS suggest endpoint) lives in its own file and is registered here with one line.

Step 6 — The risk router validates the incoming request

File: `cds-service/risk_router.py`

Why does this file exist separately from `risk_model.py`? There is a clear separation of concerns:

- `risk_router.py` owns everything about **the HTTP interface**: how is the endpoint called, what does the input look like, what HTTP errors should be raised, what does the response structure look like.
- `risk_model.py` owns everything about **the prediction logic**: model loading, feature ordering, SHAP calculation, comment generation.

This separation makes each file independently testable and replaceable. If the HTTP API contract changes (e.g., adding authentication or a different request format), only `risk_router.py` needs to change. If the model is retrained with different features, only `risk_model.py` needs updating. They are deliberately kept separate.

```
class RiskRequest(BaseModel):
    age:                        Optional[float] = None
    gender:                     Optional[float] = None
    bmi:                        Optional[float] = None
    # ... (all 54 fields, each Optional[float] = None)
    consult_count_12m:          Optional[float] = None
```

What this does: `BaseModel` is from the Pydantic library. It defines what the HTTP request body must look like. Every field is declared as `Optional[float] = None` — meaning it can be a number or absent entirely (JSON `null` or missing key). When FastAPI receives the JSON body sent by Node, it automatically:

1. Parses the JSON
2. Validates each field against this schema
3. Converts valid numbers to Python `float`
4. Converts JSON `null` to Python `None`
5. Rejects the request with an error if a field has an unexpected type

This Pydantic validation is the first defence against malformed data reaching the models.

The endpoint itself:

```
@router.post("/cds/risk-predict")
async def risk_predict(req: RiskRequest):
    try:
        patient_data = req.model_dump()   # Pydantic BaseModel → plain dict
        result = predict(patient_data)
        return {
            "status":      "ok",
            "predictions": result
        }
    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Risk prediction failed: {str(e)}"
        )
```

What this does: `req.model_dump()` converts the validated Pydantic model back to a plain Python dictionary (e.g., `{"age": 52.0, "gender": 1.0, "bmi": 28.5, "creatinine_umol": None, ...}`). This is passed to `predict()` in `risk_model.py`. If `predict()` raises any exception (model file missing, XGBoost error, etc.),

the `except` block wraps it in an HTTP 500 response with a description. FastAPI automatically serialises the returned dict to JSON.

Step 7 — The prediction engine

File: `cds-service/risk_model.py`

Model loading at startup:

```
MODELS_DIR = Path(__file__).parent / "models"

with open(MODELS_DIR / "feature_cols.json") as f:
    FEATURE_COLS = json.load(f)

model_htn = xgb.Booster()
model_htn.load_model(str(MODELS_DIR / "model_hypertension.json"))

model_cvd = xgb.Booster()
model_cvd.load_model(str(MODELS_DIR / "model_cvd.json"))

model_dm = xgb.Booster()
model_dm.load_model(str(MODELS_DIR / "model_diabetes.json"))

model_ckd = xgb.Booster()
model_ckd.load_model(str(MODELS_DIR / "model_ckd.json"))
```

What this does: These lines run once, when `uvicorn main:app` starts up. `xgb.Booster()` creates an empty XGBoost model object. `.load_model(path)` loads the trained model parameters from the JSON file on disk into that object.

Why is model loading at module level rather than inside `predict()`? Loading a model from disk takes significant time — it reads and deserialises a file that may contain thousands of decision trees. If this happened on every request, the first prediction for every clinician would take many seconds. By loading at startup, the cost is paid once. All four model objects then live in Python's memory for the lifetime of the process, ready to run predictions in milliseconds.

`Path(__file__).parent / "models"` constructs a path relative to the current file. `__file__` is Python's built-in variable for the absolute path of the current script. `.parent` strips the filename to give the containing directory. `/ "models"` adds the `models` subdirectory. This is more reliable than a hardcoded path like `"./models"` because it works regardless of which directory the server is started from.

The predict() function — building the feature matrix:

```
def predict(patient_data: dict) -> dict:
    values = [patient_data.get(col, np.nan) for col in FEATURE_COLS]
    arr = np.array(values, dtype=np.float32).reshape(1, -1)
    dmatrix = xgb.DMatrix(arr, feature_names=FEATURE_COLS)
```

What this does:

`FEATURE_COLS` is the ordered list of 54 feature names loaded from `feature_cols.json`. The list comprehension `[patient_data.get(col, np.nan) for col in FEATURE_COLS]` iterates through every name in that list in order and looks up the corresponding value in the patient dictionary. If a feature name is not present or is `None`, it uses `np.nan` (NumPy's "not a number") as the placeholder. This produces a list of 54 numbers, in exactly the right order.

A **feature vector** is this list of 54 numbers representing one patient. XGBoost thinks in terms of a table of data (rows = patients, columns = features). Even though we only have one patient, we must give it a table — `reshape(1, -1)` does this by saying "make it 1 row and however many columns are needed" (NumPy fills in 54 automatically from the array length).

`xgb.DMatrix` is XGBoost's internal data format — a **DMatrix** (Dense Matrix). It is not a plain NumPy array; it is XGBoost's own optimised structure that handles missing values (`NaN`) natively. The `feature_names=FEATURE_COLS` argument associates column names with the matrix so XGBoost can verify it receives features in the expected order.

Producing probabilities via sigmoid:

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

prob_htn = float(sigmoid(model_htn.predict(dmatrix)[0]))
prob_cvd = float(sigmoid(model_cvd.predict(dmatrix)[0]))
prob_dm  = float(sigmoid(model_dm.predict(dmatrix)[0]))
prob_ckd = float(sigmoid(model_ckd.predict(dmatrix)[0]))
```

What sigmoid means and why it's needed here: XGBoost trees internally work with *log-odds* (also called the "margin" or "raw score"). This is a number from −∞ to +∞ where larger values indicate higher risk. It is not directly a probability. To convert a log-odds score to a probability between 0 and 1, you apply the **sigmoid function**: `1 / (1 + e^(-x))`.

Some examples to make this concrete:

- Raw score of `0.0` → sigmoid → `0.5` (50% probability, maximum uncertainty)
- Raw score of `2.0` → sigmoid → `0.88` (88% probability, high risk)
- Raw score of `-2.0` → sigmoid → `0.12` (12% probability, low risk)
- Raw score of `4.0` → sigmoid → `0.98` (98% probability, very high risk)

`model_htn.predict(dmatrix)` returns an array (one result per row in the DMatrix). Since we have one patient, `[0]` extracts that single result. `float()` converts the NumPy scalar to a plain Python float.

SHAP values — what they are and how they're computed:

SHAP stands for **SHapley Additive exPlanations**, named after a concept from cooperative game theory. The core idea: for each feature, SHAP calculates "how much did including this feature in the prediction change the outcome, compared to if it had been missing?" Positive SHAP = the feature pushed the risk score up. Negative SHAP = the feature pushed the risk score down.

```
def get_shap_explanation(model, arr):
    shap_raw = model.predict(dmatrix, pred_contribs=True)[0]
    shap_features = shap_raw[:-1]   # last element is the bias term — exclude it

    pairs = list(zip(FEATURE_COLS, shap_features, values))
    pairs_sorted = sorted(pairs, key=lambda x: abs(x[1]), reverse=True)

    top3 = []
    for feat, shap_val, pat_val in pairs_sorted[:3]:
        direction = "up" if shap_val > 0 else "down"
        comment = get_comment(feat, direction, pat_val if not np.isnan(pat_val)
else 0)
        top3.append({
            "feature":   feat,
            "value":     round(float(pat_val), 2) if not np.isnan(pat_val) else
None,
            "impact":    round(float(shap_val), 3),
            "direction": "increases_risk" if shap_val > 0 else "decreases_risk",
            "comment":   comment
        })
    return top3
```

Line-by-line explanation:

- `model.predict(dmatrix, pred_contribs=True)` — `pred_contribs=True` is the XGBoost flag that activates SHAP calculation. Instead of returning one number, it returns one number *per feature* (the SHAP contribution) plus one extra number (the model's baseline bias term, representing the average prediction across all training patients). The array has 55 elements for our 54-feature model.
- `shap_raw[:-1]` — slices off the last element (the bias) to get just the 54 feature contributions.
- `zip(FEATURE_COLS, shap_features, values)` — combines three sequences into tuples: `('age', -0.12, 52.0)`, `('bp_visit_avg', 0.63, 146.25)`, etc.
- `sorted(..., key=lambda x: abs(x[1]), reverse=True)` — sorts by absolute SHAP value, largest first. A large negative SHAP is as important as a large positive SHAP; `abs()` treats them equally for ranking purposes.
- `pairs_sorted[:3]` — takes only the top three most influential features.
- `direction = "up" if shap_val > 0 else "down"` — positive SHAP means the feature pushed the risk score up (increases risk); negative means it pulled it down (decreases risk or is protective).

How a SHAP value becomes a sentence:

```
COMMENT_TEMPLATES = {
    "bp_visit_avg": {
        "up":   "Blood pressure of {value} mmHg is the primary driver of risk",
        "down": "Blood pressure of {value} mmHg is within acceptable range"
    },
    "hba1c_pct": {
        "up":   "HbA1c of {value}% reflects poor glycaemic control over the past 3
months",
        "down": "HbA1c of {value}% reflects good long-term glucose control"
```

```
    },
    # ... 20+ more entries
}

def get_comment(feature: str, direction: str, value: float) -> str:
    template = COMMENT_TEMPLATES.get(feature, {}).get(direction)
    if template:
        try:
            return template.format(value=round(value, 1))
        except Exception:
            return template
    return f"{feature.replace('_', ' ')} is {'elevated' if direction == 'up' else
'within range'}"
```

What this does: `COMMENT_TEMPLATES` is a nested dictionary. The outer key is the feature name (`"bp_visit_avg"`); the inner key is `"up"` or `"down"`. The value is a template string with `{value}` as a placeholder. `template.format( value=round(value, 1))` substitutes the patient's actual measurement in. If no template exists for a feature (the `get()` returns `None`), the fallback generates a generic sentence from the feature name by replacing underscores with spaces.

Building the clinical summary:

```
def build_summary(condition: str, prob: float, shap: list) -> str:
    level = get_risk_level(prob)
    drivers = [s["comment"] for s in shap if s["direction"] == "increases_risk"]
    protective = [s["comment"] for s in shap if s["direction"] ==
"decreases_risk"]

    if not drivers:
        return f"{condition} risk is {level} ({prob*100:.0f}%). No major risk
drivers identified."

    driver_text = drivers[0]
    if len(drivers) > 1:
        driver_text += f", and {drivers[1].lower()}"

    summary = f"{condition} risk is {level} ({prob*100:.0f}%). {driver_text}."
    if protective:
        summary += f" Note: {protective[0].lower()}."
    return summary
```

What this does: Takes the top three SHAP factors and assembles a single English sentence combining the risk level, percentage, primary driver, optional secondary driver, and one protective factor if present. The output might be: "Hypertension risk is high (72%). Blood pressure of 148 mmHg is the primary driver of risk, and sustained elevated BP averaging 146.25 mmHg across visits confirms persistent hypertension. Note: younger age is partially protective."

The risk level thresholds:

```
def get_risk_level(prob: float) -> str:
    if prob >= 0.85:   return "very high"
    elif prob >= 0.70: return "high"
    elif prob >= 0.40: return "moderate"
    else:              return "low"
```

These thresholds (40%, 70%, 85%) are **clinical decisions, not statistical outputs**. They were defined by clinicians based on what probability thresholds are meaningful in the context of each condition. They are not derived from model performance metrics.

Step 8 — The response travels back to React

The response JSON leaving FastAPI looks like:

```
{
  "status": "ok",
  "predictions": {
    "hypertension":   { "probability": 0.72, "risk_level": "high",
"top_factors": [...], "summary": "..." },
    "cardiovascular": { "probability": 0.41, "risk_level": "moderate",
"top_factors": [...], "summary": "..." },
    "diabetes":       { "probability": 0.14, "risk_level": "low",
"top_factors": [...], "summary": "..." },
    "ckd":            { "probability": 0.58, "risk_level": "high",
"top_factors": [...], "summary": "..." }
  }
}
```

Node receives this, wraps it with the `patientId` and `features`, and sends it to React. React's `fetchRisk()` stores `data.predictions` in `riskData`. The component then renders it, pulling the `summary` string for the expandable detail:

```
{isExpanded && (
  <div>
    <p style={{ fontStyle: 'italic' }}>{cond.summary}</p>  {/* one-line summary
*/}

    {cond.top_factors.map((f, i) => (
      <div key={i} style={{
        background: f.direction === 'increases_risk'
          ? 'var(--badge-red-bg)' : 'var(--badge-green-bg)',
      }}>
        <span>{f.direction === 'increases_risk' ? '↑' : '↓'}</span>
        <span>{f.comment}</span>
        <span>({f.impact > 0 ? '+' : ''}{f.impact.toFixed(2)})</span>
      </div>
    ))}
```

```
    </div>
  )}
```

What this does: When the user clicks a condition row, `isExpanded` becomes true. The summary sentence is shown in italics. Each SHAP factor is rendered as a card — red-tinted for factors that increase risk, green-tinted for protective factors. The arrow (↑ or ↓) matches the direction. The SHAP impact number (e.g., `+0.63`) shows the magnitude. This renders the `comment` field returned by `get_comment()` in the Python model.

Worked Example with real numbers

Patient: John Banda, 52-year-old male, HIV negative, attending for follow-up. He has had four blood pressure readings over the past year and one elevated glucose result. No kidney function tests on file.

Step 1 — Feature assembly in `risk.controller.ts`:

```
age:                  52      ← Math.floor((now - DOB) / year_ms)
gender:                1      ← 'MALE' → 1
bmi:                28.5      ← 86kg / 1.74m² = 28.4, rounded to 28.5
hiv_status:            0      ← 'NEGATIVE' → 0
on_art:             null      ← HIV negative, so we can't determine

bp_reading1:         148      ← most recent systolic (most recent vital row)
bp_reading2:          92      ← most recent diastolic
bp_visit_avg:     146.25      ← avg([145, 150, 142, 148])
bp_3visit_avg:    146.67      ← avg([150, 142, 148]) — last three
bp_6month_avg:    146.25      ← all readings within 6 months
bp_12month_max:      150      ← Math.max(145, 150, 142, 148)
bp_trend:            0.1      ← linSlope([145, 150, 142, 148]) — slight upward
bp_variability:     3.03      ← stddev([145, 150, 142, 148])
elevated_visit_count: 4       ← all readings ≥ 140

fasting_glucose_mmol: null    ← no fasting glucose on file
rbs_mmol:            8.5      ← from investigation result, componentId 54
hba1c_pct:          null      ← no HbA1c on file

creatinine_umol:    null      ← NO CREATININE ON FILE
egfr:               null      ← cannot compute without creatinine
urea_mmol:          null      ← no urea on file

haemoglobin_g_dl:   null      ← no FBC on file

consult_count_12m:    6       ← 6 encounters in past 12 months

waist_cm, smoking_status, fhx_*, cholesterol, liver, thyroid: all null
```

Step 2 — The feature vector sent to FastAPI:

```
{
  "age": 52, "gender": 1, "bmi": 28.5, "waist_cm": null,
  "smoking_status": null, "pack_years": null, "alcohol_units_per_week": null,
  "physical_activity_hrs_week": null, "dietary_quality": null,
  "fhx_hypertension": null, "fhx_diabetes": null, "fhx_cvd": null,
  "fhx_ckd": null, "fhx_cancer": null,
  "hiv_status": 0, "on_art": null, "prev_cvd_event": null, "gestational_diabetes":
null,
  "bp_reading1": 148, "bp_reading2": 92, "bp_visit_avg": 146.25,
  "bp_3visit_avg": 146.67, "bp_6month_avg": 146.25, "bp_12month_max": 150,
  "bp_trend": 0.1, "bp_variability": 3.03, "elevated_visit_count": 4,
  "fasting_glucose_mmol": null, "rbs_mmol": 8.5, "hba1c_pct": null,
  "total_cholesterol_mmol": null, "ldl_mmol": null, "hdl_mmol": null,
  "triglycerides_mmol": null, "non_hdl_mmol": null, "tc_hdl_ratio": null,
  "tg_hdl_ratio": null,
  "creatinine_umol": null, "egfr": null, "urea_mmol": null,
  "microalbumin_mg_l": null, "sodium_mmol": null, "potassium_mmol": null,
  "alt_u_l": null, "ast_u_l": null, "ggt_u_l": null,
  "crp_mg_l": null, "esr_mm_hr": null,
  "haemoglobin_g_dl": null, "wbc_x10_9": null, "platelets_x10_9": null,
  "tsh_miu_l": null, "uric_acid_umol": null, "consult_count_12m": 6
}
```

Step 3 — What FastAPI does with the null values:

In `predict()`: `[patient_data.get(col, np.nan) for col in FEATURE_COLS]` converts each Python `None` to `np.nan`. So all null fields become `NaN` in the NumPy array. XGBoost's DMatrix stores these internally as a sentinel value it recognises as "missing". During tree traversal, when the model reaches a split on `creatinine_umol` and the value is missing, it follows the "default direction" branch — a direction the training algorithm learned is best when that feature is absent.

Step 4 — Hypothetical model outputs:

```
Hypertension model:   raw score ≈ +2.1 → sigmoid → 0.89 → "very high" (≥0.85)
Cardiovascular model: raw score ≈ +1.1 → sigmoid → 0.75 → "high" (≥0.70)
Diabetes model:       raw score ≈ -0.5 → sigmoid → 0.38 → "low" (<0.40)
                      (elevated RBS raises it but no HbA1c or fasting glucose
                       to confirm; model stays in low range without those)
CKD model:            raw score ≈ +0.4 → sigmoid → 0.60 → "moderate" (0.40–0.70)
                      (hypertension is a CKD risk factor, but no creatinine
                       means the model cannot confirm actual kidney damage)
```

Note: these numbers are illustrative; actual outputs depend on the trained weights.

Step 5 — What the clinician sees:

```
● Hypertension      89%   [Very High]  ▸
● Cardiovascular    75%   [High     ]  ▸
```

```
● Diabetes          38%   [Low     ]  ▸
● CKD               60%   [Moderate]  ▸
```

Clicking **Hypertension** expands to:

> *Hypertension risk is very high (89%). Blood pressure of 148 mmHg is the primary driver of risk, and BP was elevated in 4 of 4 recent visits indicating uncontrolled hypertension.*

↑ Blood pressure of 148 mmHg is the primary driver of risk (+0.63)
↑ BP was elevated in 4 of 4 recent visits indicating uncontrolled hypertension (+0.41)
↓ Younger age is partially protective (−0.18)

---

3. The Four XGBoost Models in Detail

What XGBoost is, in plain language

XGBoost (Extreme Gradient Boosting) is a machine learning algorithm that builds a large number of decision trees in sequence, where each new tree corrects the mistakes of the previous ones. The final prediction is the sum of scores from all trees. For medical prediction, the input is a row of patient measurements (the feature vector) and the output is a number indicating disease risk. The model learns which combinations of features predict disease by training on historical patient data where outcomes are known.

Where the model files live and why

```
cds-service/models/
├── feature_cols.json       ← The 54-column contract between Node and Python
├── model_hypertension.json
├── model_cvd.json
├── model_diabetes.json
└── model_ckd.json
```

These files live inside `cds-service/` (the Python service directory) because they are consumed exclusively by Python code — specifically by `risk_model.py` using the `xgboost` library. Node never reads these files; Node only sends feature vectors to the HTTP endpoint. Keeping the models physically co-located with the code that uses them means they can be deployed, updated, or backed up as a unit.

The models are in `models/` (a subdirectory) rather than the root of `cds-service/` for cleanliness — the JSON model files are large binary-encoded artifacts that would clutter the directory alongside readable Python source files.

What `feature_cols.json` contains and why it needs to be a separate file

```
[
  "age", "gender", "bmi", "waist_cm", "smoking_status",
  "pack_years", "alcohol_units_per_week", "physical_activity_hrs_week",
  "dietary_quality", "fhx_hypertension", "fhx_diabetes", "fhx_cvd",
```

```
    "fhx_ckd", "fhx_cancer", "hiv_status", "on_art", "prev_cvd_event",
    "gestational_diabetes", "bp_reading1", "bp_reading2", "bp_visit_avg",
    "bp_3visit_avg", "bp_6month_avg", "bp_12month_max", "bp_trend",
    "bp_variability", "elevated_visit_count", "fasting_glucose_mmol",
    "rbs_mmol", "hba1c_pct", "total_cholesterol_mmol", "ldl_mmol",
    "hdl_mmol", "triglycerides_mmol", "non_hdl_mmol", "tc_hdl_ratio",
    "tg_hdl_ratio", "creatinine_umol", "egfr", "urea_mmol",
    "microalbumin_mg_l", "sodium_mmol", "potassium_mmol", "alt_u_l",
    "ast_u_l", "ggt_u_l", "crp_mg_l", "esr_mm_hr", "haemoglobin_g_dl",
    "wbc_x10_9", "platelets_x10_9", "tsh_miu_l", "uric_acid_umol",
    "consume_count_12m"
]
```

Why this file exists as a separate JSON rather than being hardcoded in `risk_model.py`:

XGBoost models are order-sensitive. The model was trained with features presented in a specific sequence (column 0 = age, column 1 = gender, ...). When making a prediction, the input array must have the same feature in the same position. `risk_model.py` uses this list to arrange the incoming dictionary into the correct array order:

```
values = [patient_data.get(col, np.nan) for col in FEATURE_COLS]
```

If `feature_cols.json` did not exist and the order were hardcoded, changing it would require editing Python source code and redeploying. As a JSON file, the training pipeline and the serving pipeline can share the same file — the training script that produced the model can write `feature_cols.json` automatically, and the serving code reads it. If the model is retrained with one feature removed or two features reordered, only the JSON file changes; the Python prediction code needs no modification.

The four models — what each predicts

`model_hypertension.json` — predicts the probability that this patient will develop or already has sustained hypertension. Features with the most training signal: BP readings, BP trend, count of elevated visits, age, BMI.

`model_cvd.json` — predicts cardiovascular disease risk (heart attack, stroke, coronary artery disease). Features with most signal: BP, cholesterol and HDL ratios, age, sex, HIV status, BMI, smoking, CRP (inflammation marker). This model benefits most from lipid panel results; without them, most of its strongest features are missing.

`model_diabetes.json` — predicts type 2 diabetes risk. Features with most signal: fasting glucose, HbA1c, BMI, age, family history, gestational diabetes history. RBS from vitals provides weak signal when HbA1c is absent.

`model_ckd.json` — predicts chronic kidney disease risk. Features with most signal: creatinine, eGFR, microalbumin, BP (a leading cause of kidney damage), diabetes markers (another cause), urea, sodium, potassium.

Do the inputs from `risk.controller.ts` actually match what the models expect?

Yes, by design. The Node controller was written with knowledge of exactly what features the models were trained on. The `COMPONENT_MAP` was constructed by identifying which investigation catalogue component IDs produce values for which model features. The encoding choices (FEMALE=0, MALE=1; POSITIVE=1, NEGATIVE=0) were made to match the training data encoding.

The contract is enforced through `feature_cols.json`: as long as Node sends a dictionary with these 54 field names containing numbers or `null`, the prediction will work correctly regardless of which fields are missing.

---

4. Known Issues and Footguns

Issue 1 — Stale Prisma client causes silent data loss on `order.items`

File: `server/src/controllers/risk.controller.ts`, line 208

The code:

```
const orders = await prisma.investigationOrder.findMany({
  where: { patientId },
  include: {
    items: {
      include: {
        results: { orderBy: { resultedAt: 'asc' } },
      },
    },
  },
});

for (const order of orders) {
  for (const item of (order as any).items ?? []) {   // ← defensive guard added
```

What goes wrong: The Prisma client is a code-generated file in `server/node_modules/.prisma/client/` that is produced by running `npx prisma generate`. If the database schema changes (e.g., when `InvestigationOrderItem` was first added) and `prisma generate` is not run, the generated client does not know about the `items` relation. At runtime, `order.items` would be `undefined` instead of an array. The original code `for (const item of order.items)` would then throw `TypeError: undefined is not iterable`, caught by the outer `try/catch`, which would return a response without an `error` field — causing the React frontend to show the generic fallback message "Risk prediction service unavailable" rather than the actual error.

Consequence for a clinician: They see a red error message in the risk panel and no risk scores. Because the error message is generic, there is no way to tell from the UI that lab results are simply not being read. The models would still produce scores, but from demographic and vital data only — if the clinician doesn't see the panel at all (due to the error), they miss all scores entirely.

The fix applied: `(order as any).items ?? []` adds a defensive fallback: if `items` is undefined (stale client), the inner loop simply does not execute, which means no lab features are populated. The models run with those fields as `null`, which is the correct behaviour for "no lab data on file." The real fix is running `npx prisma generate` when the server is not running.

Issue 2 — JSON serialisation silences errors when `err.message` is undefined

File: `server/src/controllers/risk.controller.ts`, original catch block

The original code:

```
} catch (err: any) {
  res.status(500).json({ error: err.message });
}
```

What goes wrong: `JSON.stringify` silently drops keys whose value is `undefined`. If the caught error is an unusual object (e.g., a Prisma validation error) whose `.message` property is `undefined`, then `{ error: undefined }` serialises to `{}` — an empty JSON object. The React frontend receives a 500 response with an empty body, finds no `error` key, and shows the generic fallback "Risk prediction service unavailable" instead of the actual error description.

Consequence for a clinician: The UI shows a vague error. Neither the clinician nor the developer can tell what went wrong without reading server logs.

The fix applied:

```
} catch (err: any) {
  const msg = err?.message ?? String(err);
  console.error('[risk] unhandled error:', msg, err?.stack ?? '');
  res.status(500).json({ error: msg });
}
```

`err?.message ?? String(err)` falls back to `String(err)` — which converts any object to a string representation, never returning `undefined`. `console.error` with the stack trace ensures the exact error is always visible in the Node.js terminal, regardless of what the HTTP response contains.

Issue 3 — `elevated_visit_count` of zero becomes null, affecting risk scoring

File: `server/src/controllers/risk.controller.ts`, line 168

The code:

```
const elevated_visit_count = systolics.filter((s) => s >= 140).length || null;
```

What goes wrong: `0 || null` evaluates to `null` in JavaScript because `0` is falsy. This means a patient who has had three blood pressure readings and every single one was below 140 (normal) is treated identically to a patient with no blood pressure readings at all. The model cannot distinguish "this patient has been measured many times and is always normal" from "this patient has never been measured". The former is meaningful negative evidence; the latter is simply missing data.

Consequence for a clinician: The hypertension model may produce a slightly higher risk estimate for a well-monitored normotensive patient than intended, because the model cannot use the "zero elevated readings" as a protective factor. This is a subtle bias that affects only patients with BP consistently below 140 — a common presentation.

The correct fix (not yet applied):

```
const elevated_visit_count = systolics.length > 0
  ? systolics.filter((s) => s >= 140).length
  : null;
```

This sends `0` as a genuine zero count when readings exist, and `null` only when no readings exist at all.

Issue 4 — `rbs_mmol` can be populated from a vitals glucose field recorded in an unknown unit

File: `server/src/controllers/risk.controller.ts`, lines 177–181 and 270

The code:

```
let rbs_from_vitals: number | null = null;
for (let i = vitals.length - 1; i >= 0; i--) {
  if (vitals[i].glucose != null) { rbs_from_vitals = vitals[i].glucose as number;
break; }
}

const rbs_mmol: number | null = labFeatures['rbs_mmol'] ?? rbs_from_vitals;
```

What goes wrong: The `Vital` table's `glucose` column has no unit enforcement. A nurse entering a glucose reading of `8.5` likely means 8.5 mmol/L (the unit used in Southern and Eastern Africa). A nurse entering `153` likely means 153 mg/dL (the unit used in the US). Both are valid ways of expressing the same measurement, but the model expects mmol/L. There is currently no unit stored alongside the glucose value in the vitals table.

Consequence for a clinician: If glucose is entered in mg/dL (e.g., `153`), the model receives `153` where it expects `8.5`. A value of `153 mmol/L` is medically impossible and would push the diabetes model to predict extreme risk. The diabetes and hypertension scores for that patient would be dangerously inflated.

The partial mitigation in place: Lab investigation results use `COMPONENT_MAP` with explicit unit conversions. The `rbs_mmol` feature prefers a formal lab result (`labFeatures['rbs_mmol'] ?? rbs_from_vitals`) and only falls back to the vitals glucose field if no lab result exists. Until the vitals table enforces units or the data entry UI enforces mmol/L, this risk remains.

Issue 5 — `bp_reading2` is the diastolic of the most recent *vital record*, not the most recent *BP measurement*

File: `server/src/controllers/risk.controller.ts`, line 161

The code:

```
const bp_reading2 = vitals.at(-1)?.bloodPressureDiastolic ?? null;
```

What goes wrong: `vitals.at(-1)` is the last vital record by date. If a nurse records weight and temperature on Day 1 (without taking BP), and the last BP was recorded on Day 0, then `vitals.at(-1)` points to Day 1 — which has `bloodPressureDiastolic = null`. The diastolic would then be `null`, even though there is a valid recent diastolic reading one row earlier.

Compare with `bp_reading1`:

```
const bp_reading1 = systolics.at(-1) ?? null;
```

This correctly takes the last element of `systolics` — which has already been filtered to exclude null values. `bp_reading2` does not apply the same filtering.

Consequence for a clinician: For patients who regularly have some vitals recorded without full BP measurement, `bp_reading2` (diastolic) would frequently be `null` even when there is valid diastolic data in the record. The models trained on this feature expect diastolic when available; an unexpected `null` introduces noise into the hypertension and cardiovascular predictions.

The correct fix:

```
const diastolics = vitals
  .map((v) => v.bloodPressureDiastolic)
  .filter((v): v is number => v != null);
const bp_reading2 = diastolics.at(-1) ?? null;
```